

Disease Prediction & Reasoning Website

Project Team Framework & Role Assignments

Project Goal: Build a full-stack web application where farmers can upload plant leaf images to identify diseases. crucial feature: provide 'Reasoning', Symptoms, and Cure recommendations, not just the class prediction.

Dataset: PlantVillage (Tomato, Potato, Pepper, etc.)

Total Roles: 5

Role 1 : Data Cleaning, EDA & Preprocessing

Assignee: Data Engineer / Analyst

Objective: Prepare the raw PlantVillage dataset to ensure high quality input for the ML model.

Tech Stack:

- Python
- Pandas, NumPy
- OpenCV (cv2)
- Matplotlib / Seaborn

Key Responsibilities:

1. Structure Fix: Consolidate the nested 'PlantVillage' folders into a single 'dataset/train' directory.
2. Quality Check: Remove corrupt images and check for class imbalance (e.g., if 'Tomato_Healthy' has 2000 images but 'Potato_Early_Blight' has only 200).
3. Preprocessing:
 - Resize all images to a fixed standard (e.g., 224x224).
 - Normalize pixel values (0-255 -> 0-1).
4. Augmentation: Generate synthetic data (rotation, flip, zoom) to balance smaller classes.
5. Splitting: Create Train (70%), Validation (15%), and Test (15%) sets.

Deliverable: A clean, split dataset ready for training.

Role 2 : Machine Learning Model & Reasoning

Assignee: AI/ML Engineer

Objective: Build a Deep Learning model to classify diseases and map them to explanations.

Tech Stack:

- Python
- TensorFlow/Keras or PyTorch
- Jupyter Notebook

Key Responsibilities:

1. Model Architecture: Use Transfer Learning (Recommended: MobileNetV2 for speed or ResNet50 for

Disease Prediction & Reasoning Website

accuracy).

- Do NOT build from scratch unless necessary.
- 2. Training: Train on the cleaned dataset. Use Categorical Crossentropy loss and Adam optimizer.
- 3. Reasoning System (CRITICAL):
 - Create a JSON/Dictionary mapping class names to human-readable explanations.
 - Example: {'Tomato_Early_blight': {'cause': 'Fungal infection...', 'cure': 'Use copper fungicide...'}}
 - OPTIONAL: Integrate a small LLM (Llama/Gemini) to generate dynamic reasoning.
- 4. Export: Save the trained model (.h5/.pt) and the class_indices mapping.

Deliverable: 'model.h5' and 'reasoning_map.json'.

Role 3 : Backend Development (API)

Assignee: Backend Developer

Objective: Serve the ML model to the web and manage application logic.

Tech Stack:

- Python (FastAPI)
- Uvicorn

Key Responsibilities:

1. API Setup: Initialize a FastAPI application.
2. Prediction Endpoint (/predict):
 - Accept POST request with image file.
 - Preprocess image (same logic as ML team).
 - Run model inference.
 - Retrieval: Look up the predicted class in the 'Reasoning Mapping' to get Cause and Cure.
 - Return JSON: {prediction, confidence, reasoning, cure}.
3. Feedback Loop: Create an endpoint to save user feedback on prediction accuracy.
4. CORS: Enable CORS to allow the Frontend to connect.

Deliverable: A running API server.

Role 4 : Frontend Development (UI/UX)

Assignee: Frontend Developer

Objective: Create a modern, responsive, and visually stunning user interface.

Tech Stack:

- Next.js (React Framework)
- Tailwind CSS (Styling)
- Framer Motion (Animations)

Key Responsibilities:

1. Design:
 - Theme: Nature/Agriculture (Greens, Earthy tones) but Modern (Glassmorphism).
 - Hero Section: 'AI Doctor for your Crops'.

Disease Prediction & Reasoning Website

2. Upload Feature: Drag & Drop zone with image preview.
3. Results Display:
 - Show the detected disease clearly.
 - Display the 'Reasoning' and 'Cure' in distinct, easy-to-read cards.
4. Loading State: Smooth animation while waiting for the model.

Deliverable: A deployed web application.

Role 5 : Quality Assurance & Testing

Assignee: QA Engineer

Objective: Ensure reliability, accuracy, and user experience.

Tech Stack:

- pytest
- Postman
- Jest/Cypress

Key Responsibilities:

1. Model Testing: Evaluate model on the 'Test Set'. Check for False Positives.
2. API Testing: Ensure the backend handles large images or invalid files gracefully.
3. UI Testing: Check responsiveness on Mobile, Tablet, and Desktop.
4. Integration: Verify that the frontend correctly displays data sent by the backend.

Deliverable: Test report and Bug list.